

Taller 4 - Máquinas de estados finitos

Sistemas Digitales

Segundo Cuatrimestre 2026

Controlador de un portón automático, en SystemVerilog.

1. Enunciado

Vamos a diseñar, primero en papel y luego en SystemVerilog, el controlador de un portón automático. Los primeros tres ejercicios agregan progresivamente comportamiento al mismo sistema. El cuarto ya está resuelto: hay que simularlo y comparar, sobre la forma de onda, los dos estilos de máquina de estados (Moore y Mealy).

Usar el paquete inicial del Taller 4 (devcontainer, esqueletos `.sv` y testbenches) de la página de la materia.

2. Cómo trabajar

Para los ejercicios 1 a 3:

1. Identificar las entradas, las salidas y los estados necesarios.
2. Dibujar el diagrama de estados en papel.
3. Indicar sobre el diagrama qué condición produce cada transición.
4. Determinar el valor de las salidas en cada estado.
5. Recién entonces traducir el diseño a SystemVerilog.

Los archivos `top_module.sv` y `async_button.sv` están provistos y no deben modificarse. El `top_module` conecta el botón con el controlador y transforma las salidas del circuito en luces para poder observar su funcionamiento. En el ejercicio 4, `detector_mealy.sv` y `detector_moore.sv` también están provistos: no hay que completarlos.

En la interfaz, un toque del botón se representa mediante un pulso breve:

`button: 0 — 1 — 0`

El módulo `async_button` conserva ese toque hasta que la máquina pueda leerlo en un flanco ascendente de `clk`. Por lo tanto, cada toque debe producir una única transición de la máquina de estados.

Al activar `rst`, todos los ejercicios deben regresar al estado inicial, con el portón cerrado.

Importante: Los testbenches son parte de la especificación: no deben modificarse.

3. Ejercicios

3.1. Ejercicio 1: Control básico del portón

Diseñar una máquina de cuatro estados —CERRADO, ABRIENDO, ABIERTO, CERRANDO— que empiece en CERRADO y controla un portón:

- El botón solo se atiende en los estados extremos: en CERRADO inicia la apertura y en ABIERTO inicia el cierre. Un toque durante ABRIENDO o CERRANDO no tiene ningún efecto.
- El movimiento no depende del botón: tras un ciclo en ABRIENDO el portón queda ABIERTO, y tras un ciclo en CERRANDO queda CERRADO.

Cada estado enciende su propia luz (`led_cerrado`, `led_abriendo`, `led_abierto`, `led_cerrando`); exactamente una debe estar encendida en todo momento.

```
module porton_fsm (
    input  logic clk,
    input  logic rst,
    input  logic button,
    output logic cerrado,
    output logic abriendo,
    output logic abierto,
    output logic cerrando
);
    // completar: FSM Moore con estados CERRADO, ABRIENDO, ABIERTO y CERRANDO.
endmodule
```

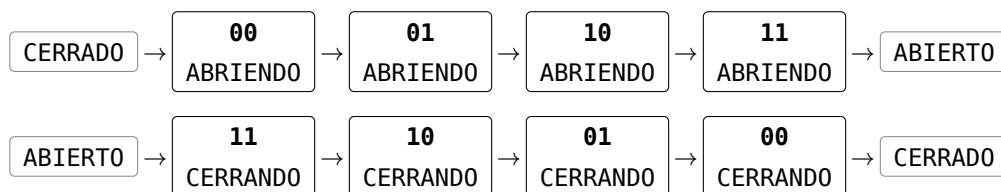
Completar `porton_fsm.sv`.

3.2. Ejercicio 2: Un portón que tarda en moverse

Extender el ejercicio anterior para que el portón tarde varios ciclos en moverse. Su posición se representa con un valor de 2 bits: 00, 01, 10, 11.

El botón sigue disparando la apertura o el cierre solo desde los extremos (posición 0 o 3), e ignorándose durante el movimiento. Mientras la puerta se este en estado ABRIENDO o CERRANDO, la posición cambia una unidad por cada flanco de clock en la dirección correspondiente, y el movimiento se detiene solo al llegar al otro extremo (0 o 3). No por cuenta de ciclos fijos como en el ejercicio 1.

Esto significa que el portón se mueve de la siguiente manera. Cada paso de ABRIENDO o CERRANDO corresponde a una posición de 2 bits; al llegar al otro extremo queda ABIERTO o CERRADO:



Es decir, tarda 4 ciclos en ABRIENDO o CERRANDO.

Las salidas deben mostrar:

Salida	Cuándo
<code>led_abriendo</code> / <code>led_cerrando</code>	hay una orden de mover la posición en esa dirección
<code>led_abierto</code> / <code>led_cerrado</code>	el portón está detenido en ese extremo
<code>luces_posicion</code>	posición actual, entre 0 y 3

```

module porton_fsm_long (
    input  logic      clk,
    input  logic      rst,
    input  logic      button,
    input  logic [1:0] posicion,
    output logic      subir,
    output logic      bajar,
    output logic      abierto,
    output logic      cerrado
);
    // completar: FSM Moore que controla el contador según la posición 0..3.
endmodule

module contador_posicion (
    input  logic      clk,
    input  logic      rst,
    input  logic      subir,
    input  logic      bajar,
    output logic [1:0] posicion
);
    // completar: contador saturado entre 0 (cerrado) y 3 (abierto).
endmodule

```

Completar porton_fsm.sv y contador_posicion.sv.

Nota: La implementación interna puede organizarse de distintas maneras, pero debe respetar este comportamiento observable.

3.3. Ejercicio 3: Pausa e inversión del movimiento

Agregar al portón del ejercicio anterior la posibilidad de detenerlo durante el movimiento e invertir luego su dirección, usando el mismo botón. El comportamiento desde los extremos no cambia.

En este caso, un toque durante el movimiento deja el portón pausado en la posición intermedia en la que se encuentre, y el toque siguiente invierte la dirección (si estaba abriendo, empieza a cerrar, y viceversa). Mientras está pausado, la posición se conserva; si esa posición resulta ser alguno de los extremos (0 o 3), el portón queda directamente CERRADO o ABIERTO en vez de pausado.

Importante: El controlador y el contador actualizan sus registros en el mismo flanco de clock. Por eso, cuando se reconoce un toque durante el movimiento, el contador completa en ese flanco el paso que ya tenía ordenado y luego queda detenido. Por ejemplo, si el portón se estaba abriendo en la posición 1, el toque puede dejarlo pausado en la posición 2.

Además de las luces del ejercicio 2, `led_pausa` debe encenderse cuando el portón está detenido en una posición intermedia.

```
module porton_fsm_smart (
    input logic      clk,
    input logic      rst,
    input logic      button,
    input logic [1:0] posicion,
    output logic      abrir,
    output logic      cerrar,
    output logic      pausa,
    output logic      abierto,
    output logic      cerrado
);
    // completar: un click inicia; durante el movimiento pausa; desde la
    // pausa otro click inicia el movimiento en la dirección opuesta.
endmodule
```

Completar `porton_fsm.sv` y `contador_posicion.sv`.

3.4. Ejercicio 4: Moore vs. Mealy: mismo detector, distinto instante

Este ejercicio no hay que implementarlo: los detectores ya están resueltos en `detector_mealy.sv` y `detector_moore.sv`. El objetivo es analizar cómo se comportan una máquina de Mealy y una de Moore sobre el mismo problema.

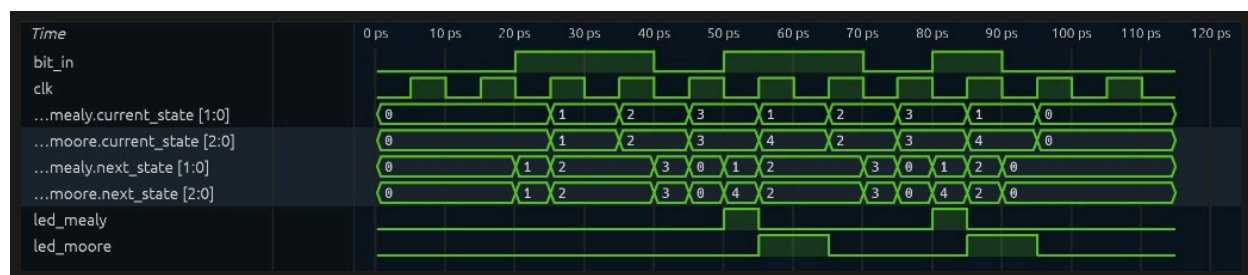
Ambas detectan la subsecuencia **1101** en un flujo serie `bit_in` (un bit por flanco de clock, con superposición permitida: en `...1101101...` el patrón se detecta dos veces).

- `led_mealy` es una salida Mealy: depende del estado actual y de `bit_in` en el mismo ciclo, por lo que puede activarse en el mismo ciclo en que llega el bit que completa el patrón.
- `led_moore` es una salida Moore: depende únicamente del estado actual, por lo que necesita al menos un estado adicional dedicado a sostener la salida durante el ciclo siguiente.

Ambos módulos comparten la misma interfaz y detectan exactamente las mismas ocurrencias del patrón; lo único que difiere es el instante exacto de la salida (y la cantidad de estados).

3.4.1. Análisis de la forma de onda

Correr `make wave` desde `ejercicios/ej4` y abrir `obj_dir/sim.vcd` en Surfer. Agregar `bit_in`, `clk`, `led_mealy` y `led_moore` a la vista. Si no se puede abrir Surfer, usar la traza de la figura.



Traza de make wave sobre el flujo 1 1 0 1 1 0 1 0.

El testbench aplica `1 1 0 1 1 0 1 0`. Ese flujo contiene **1101** en las posiciones 1–4 y, superpuesto, en las posiciones 4–7.

Analizar la forma de onda y responder:

1. Identificar cada detección de **1101** (incluido el solapamiento).
2. Medir, con los cursores de Surfer o sobre la figura, la distancia entre un pulso de `led_mealy` y el pulso correspondiente de `led_moore`. Confirmar que la diferencia es de exactamente un ciclo de clock.
3. Explicar en una oración por qué ese corrimiento es inevitable dado cómo se define una salida Moore.